SLIIT UNI
THE KNOWLEDGE UNIVERSITY

Sri Lanka Institute of Information Technology

Faculty of Computing

SE2020 - Web Technologies and Mobile

Ms. Dilani Lunugalage

BSc (Hons) in Information Technology

Lecture 05

More on Manipulating the DOM

Manipulating the DOM

- as seen last week, we can use JavaScript to access elements in the DOM
  - retrieve and store a reference to the element

```
const txtIn = document.getElementById("in");
const txtOut = document.getElementById("out");
```

- get the value of field using the value attribute

```
let input = txtIn.value;
```

- display results using the innerText attribute

```
txtOut.innerText = "result";
```

- we can also manipulate other attributes of the element

How to access DOM elements

- every element in the DOM can be accessed using JavaScript
- we have seen how to do this for elements with a unique ID:

```
const element1 = document.getElementById("id");
```

- can also be done using querySelector()
  - returns first element that matches the criteria

```
const element2 = document.querySelector("#id");

const element3 = document.querySelector(".class");

//paragraph inside div
const element4 = document.querySelector("div p");
```

querySelector() - CSS

```
<button onClick="checkVal()">Click Me</button>
<p class="para">Hello World</p>
<h3 class="para">Hello World</h3>
<script>
Function checkVal(){
const element3 = document.querySelector(".para");
paragraph.textContent = "Button Clicked!"
}
</script>
```

- → selects by **class**
- querySelector() returns the **first matching element**

How to work with attribute nodes

once required element has been retrieved you can modify **attribute** node using:

- `getAttribute(<attribute>)`
  - return the value of the attribute
- `setAttribute(<attribute>, <value>)`
  - if attribute exist, overwrites its value, otherwise, add attribute-value pair to the element
- `removeAttribute(<attribute>)`
  - delete attribute from element
- `hasAttribute(<attribute>)`
  - if attribute exists, returns true, otherwise, returns false

TO DO: change attribute nodes

- what is the effect of these JavaScript statements? Why?

```
const pic = document.querySelector("img");

console.log(pic.hasAttribute("src"));
console.log(pic.getAttribute("src"));
```

- Changing the attribute value

```
pic.setAttribute("src", "images/happy.jpg");
console.log(pic.getAttribute("src"));
```

Changing CSS rules

- can easily change appearance of elements by adding, changing or removing `class` attribute
  - get element to be targeted
  - use `setAttribute` to add or change `class` attribute
  - use `removeAttribute` to remove `class` attribute

```
element1.setAttribute("class", "cssRule");
element2.removeAttribute("class");
```

TO DO: change class attribute nodes

- what is the effect of these JavaScript statements? Why?

```
let first = document.getElementById("first");
first.removeAttribute("class");

let second = document.getElementById("second");
second.setAttribute("class", "blue");
```

Retrieving multiple elements

- examples so far have only retrieved a single element using:
  - `getElementById()`
  - `querySelector()`

- can also retrieve multiple elements using:
  - `getElementsByTagName()`
  - `getElementsByName()`
  - `getElementsByClassName()`

  } live node list

  - `querySelectorAll()`

  } non-live node list

Accessing elements

- get all matching elements. Check this using console.log

```
const allParas = document.querySelectorAll("p");
```

- access single element specifying its **index**

```
allParas[0].setAttribute("class", "green");
```

- loop through all elements using for loop

```
for (let i = 0; i < allParas.length; i++) {
    allParas[i].setAttribute("class", "blue");
}
```

Live lists and non-live lists

- all these methods return node-list of objects
  - can access individual elements
  - can loop through elements
  - but there are no methods to add new elements or remove elements
- `querySelectorAll()` returns a **non-live** list
  any changes to the DOM do not modify the original node list returned

```
document.querySelectorAll(".card.active")
document.querySelectorAll("#container > p")
document.querySelectorAll("ul li:first-child")
document.querySelectorAll("input[type='text']")
```

Live List and Non-Live list

```
let live = document.getElementsByClassName("box");
let staticList = document.querySelectorAll(".box");

console.log("Live:", live.length);
console.log("Static:", staticList.length);
```

Now add this code through dev tools

```
<div class="box"></div>
```

Live list updated instantly but not the static list

TO DO: access multiple elements

- what is the effect of these JavaScript statements? Why?

```
let x = document.getElementsByTagName("p")
for (let i = 0; i < x.length; i+=2)
{
    x[i].setAttribute("class", "green");
}
let y = document.querySelectorAll("div p");
for (let i = 0; i < y.length; i++) {
    y[i].setAttribute("class", "green");
}
let z = document.querySelectorAll("div, p");
for (let i = 0; i < z.length; i++) {
    z[i].setAttribute("class", "green");
}
```

Add Event Listeners

- We have used DOM events previously by adding listeners directly in the html element.
- However, in order to totally separate the HTML from the JavaScript code, it is better practice to add the event Listener inside the JavaScript code by using the addEventListener() method.
- This improves separation of concerns, readability and allows you to add the event listeners and effects even if you do not have control over the HTML code.
- The addEventListener() method attaches an event handler to a specified element.
- It will not overwrite existing event handlers on that element, so it is possible to add many event handlers to one element, including more than one identical event handler (e.g. you can add two click event listeners to the same element).

- The other advantage of using the addEventListener() method, is that you get control of the event bubbling (how the event propagates through the DOM nodes).
- You can remove an event listener by using the removeEventListener() method.

The addEventListener() method

Syntax

```
element.addEventListener(event, function, useCapture);
```

Where:

- `element` is the element on which we add the eventListener
- `event` is the event that we are listening for
- `function` is the function that gets called when the event is triggered. This can be a defined function or an anonymous function.
- `useCapture` is a boolean that specifies whether we use the event bubbling (false, also default option) or event capturing (true). This parameter is optional.

Example – Change Background color

```
<div id="turnPurple" style="height:150px; width:150px; border: solid thin black; margin: 10p
</div>
<div id="clearPurple" style="height:150px; width:150px; border: solid thin black; margin: 10
</div>
<script>
document.getElementById( "turnPurple" ).addEventListener( "click", function() {
        document.getElementById( "turnPurple" ).style.backgroundColor = "purple";
});
document.getElementById( "clearPurple" ).addEventListener( "click", function() {
        document.getElementById( "turnPurple" ).style.backgroundColor = "white";
});
</script>
```

Click Me I'll turn purple

Click Me to clear purple

Example 1 – Simple Counter

Click Counter

Click Counter

Click Me

7

```
<h2>Click Counter</h2>
<button id="btn">Click Me</button>
<div id="count">0</div>

<script>
    const button = document.querySelector("#btn");
    const counterDisplay = document.querySelector("#count");

    let count = 0;

    button.addEventListener("click", function() {
        count++;
        counterDisplay.textContent = count;
    });
</script>
```

Example 2 –Character Count

Live Character Counter

Live Character Counter

hi there,

Characters: 10

```
<h2>Live Character Counter</h2>
<textarea id="textBox"></textarea>
<div id="charCount">Characters: 0</div>

<script>
    const textBox = document.querySelector("#textBox");
    const charCount = document.querySelector("#charCount");

    textBox.addEventListener("input", function(event) {
        charCount.textContent =
            "Characters: " + event.target.value.length;
    });
</script>
```

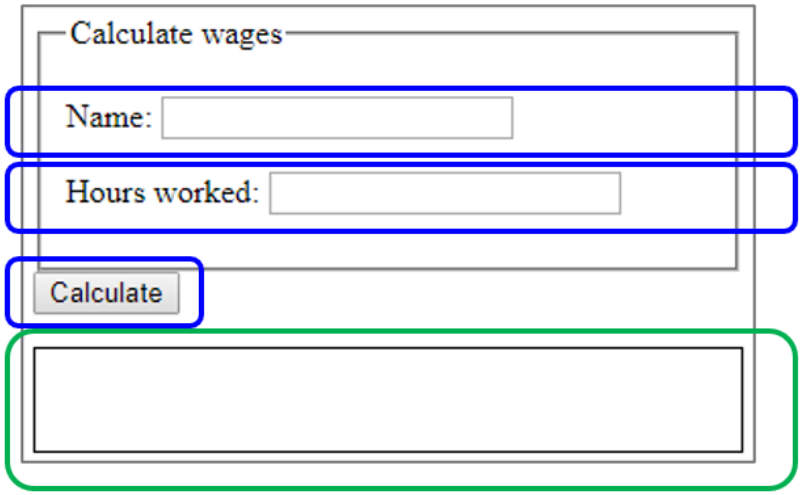
Example 03 – add form to index.html

- add following under `<!-- form -->` comment in index.html

```html
<!-- form -->
<form method="get">
    <fieldset>
        <legend>Calculate wages</legend>
        <p>
            <label for="name">Name:</label>
            <input type="text" id="name" name="name">
        </p>
        <p>
            <label for="hours">Hours worked:</label>
            <input type="number" id="hours" min="0" name="hours">
        </p>
    </fieldset>
    <button type="button" id="calculate">Calculate</button>
</form>
```

identify interactive elements

- user can interact with:
  - text field with `id=name`
  - text field with `id=hours`
  - button with `id=calculate`
  - `Hourly rate = Rs.550`
- program can output values to:
  - paragraph with `id=output`
- need to listen for events:
  - click `calculate` button



TO DO: get reference , add event listener

- First in your code in `main.js` we are getting the references to the interactive elements

```
const btnCalculate = document.getElementById("calculate");
```

- Next append the following code to `main.js` to listen for when the user clicks the **btnCalculate** and execute **calculate** when they do

```
//add event listener
btnCalculate.addEventListener("click", calculate);
```

TO DO: implement event handler

- append the following code to `main.js` to handle the click event on `btnCalculate`

```
//implement calculate event handler
function calculate() {
    console.log("calculate");
}
```

- refresh `index.html` in browser
  - click calculate button to check that calculate is called correctly
  - calculate should be displayed in the Console window each time the button is clicked

TO DO: encode calculate

- calculate function with the following:
- One line is missing , complete that line as well.

```
const hourlyRate= 550;
const name= document.getElementById("name").value;
let hours = parseInt(document.getElementById("hours").value);
const takeHome=hourlyRate * hours;
txtOutput.innerText = `${name} worked ${hours} hours
    Take home pay is: £${takeHome.toFixed(2)}`;
```

- refresh `index.html` in browser
  - enter data in the required fields and click the calculate button

Thank You!